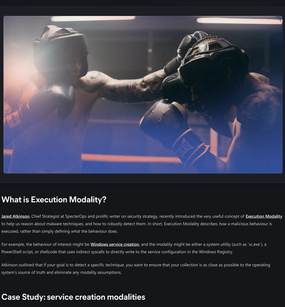
12 MAY 2025 / JOSH SALMAN

Misbehaving Modalities: Detecting Tools, Not Techniques

We explore the concept of Execution Modality and how modality-focused detections can complement behaviour-focused ones.

7 min read | Detection Engineering, Internals



What is Execution Modality?

Jared Atkinson, Chief Strategist at SpecterOps and prolific writer on security strategy, recently introduced the very useful concept of **Execution Modality** to help us reason about malware techniques, and how to robustly detect them. In short, Execution Modality describes how a malicious behaviour is executed, rather than simply defines the behaviour itself.

For example, the behaviour of remote origin is **Windows service creation**, and the modality might be either a system utility (such as `sc.exe`), a PowerShell script, or shellcode that uses indirect syscalls to directly write to the service configuration in the Windows Registry.

Atkinson outlined that if your goal is to detect a specific technique, you want to ensure that your collection is as close as possible to the operating system's source of truth and eliminate any modality assumptions.

Case Study: service creation detections

Service creation operation flow graph

In the typical Service creation scenario within the Windows OS, an installer calls `CreateServiceW`, which makes an `RPCrecvfromServer`. RPC call to an endpoint in the **Service Control Manager** (SCM, aka `services.exe`) which then makes syscalls to the **kernel mode configuration manager** to update the **database of installed services** in the registry. This is later flushed to disk and restored from disk on boot.

This means that the source of truth for running system is **the registry** (though hives are flushed to disk and can be tampered with offline).

In a threat hunting scenario, we could easily detect anomalous `sc.exe` command lines - but a different tool might make Service Control RPC calls directly.

If we were processing our threat data stringently, we could also detect anomalous Service Control RPC calls, but a different tool might make syscalls directly to our service source, such as the **Remote Registry** to update the service database instead.

In other words, some of these execution modalities bypass traditional OS logging such as **Windows event logs**.

So how do we monitor threats in the configuration manager? We can't robustly monitor syscalls directly due to **Kernel Patch Protection**, but Microsoft has provided **configuration manager callbacks** as an alternative. This is where Elastic has **focused our service creation detection** efforts - as close to the operating system source of truth as possible.

The trade-off for this low-level visibility however, is a potential reduction in context. For example, due to Windows architectural decisions, security vendors do not know which RPC client is requesting the creation of a registry key in the service database. Microsoft only supports querying RPC client details from a open active RPC service.

Starting with Windows 10 21H1, Microsoft begun invoking **RPC client details in the service creation event log**. This event, while less robust, sometimes provides additional context that might assist in determining the source of an anomalous behaviour.

Due to their history of abuse, some modalities have been extended with extra logging - one important example is PowerShell. This allows certain techniques to be detected with high precision - but only when executed via PowerShell. It is important not to conflate having detection coverage of a technique in PowerShell with coverage of that technique in general. This nuance is important when estimating **MITRE ATT&CK** coverage. As red teams increasingly develop custom C2 implants in languages other than C# / .NET for PowerShell - a close to 0% real-world coverage.

Summoning the Pyramid (STP) is a related analytic scoring methodology from MITRE. It makes a similar conclusion about the fragility of **PowerShell scriptblock-based detections** and assigns such rules a low robustness score.

High-level telemetry sources, such as Process Creation logging and PowerShell logging, are extremely brittle at detecting most techniques as they cover very few modalities. At best, they tend to be detecting the major syntaxes living off the land (LotL) abuses.

Atkinson made the following active observation in the **example** used to motivate the discussion.

An important point is that our higher-order objective in detection is behavior based, not modality based. Therefore, we should be interested in detecting Session Enumeration (behavior focused), not Session Enumeration in PowerShell (modality focused).

Sometimes that is only half of the story though. Sometimes detecting that the tool itself is out of context is more efficient than detecting the technique. Sometimes the execution modality itself is anomalous.

An alternative to detecting a known technique is to detect a misbehaving modality.

Call stacks divulge Modality

One of Elastic's strengths is the inclusion of call stacks in the majority of our events. The level of call provenance detail greatly assists in determining whether a given activity is malicious or benign. Call stack summaries are often sufficient to divulge the execution modality - the runtimes for PowerShell, .NET, RPC, WMI, VBA, Lua, Python, and Java all leave behind distinctive call stacks.

Some of our **first call stack based rules** were for Office VBA macros (`9ef4f413`) spawning child processes or dropping files, and for unbacked executable memory loading the .NET runtime. In both of these examples, the technique itself was largely benign; it was the modality of the behaviour that was predominantly anomalous.

So can we flip the typical behaviour-focused detection approach to a modality-focused one? For example, can we detect solely on the use of **any** dual-purpose API call originating from PowerShell?

Using call stacks, Elastic is able to differentiate between the API calls that originate from PowerShell scripts and those that come from the PowerShell or .NET runtimes.

Using Threat Intelligence ETW as an approximation for a dual-purpose API, our rule for "Suspicious API Call from a PowerShell Script" was quite effective.

```
api where
event.provider == "Microsoft-Windows-Threat-Intelligence" and
process.name.lower == "powershell.exe" and process.parent.name.lower == "powershell.exe"

// PowerShell Script (ps1) and interactive (ps1 equivalent)
process.thread.Ext.call_stack_final_user_module name == "System.ni.dll" and
process.thread.Ext.call_stack_final_user_module protection.provenance == "C:\Windows" and

// Suspicious memory allocation with a base address
api where true as seq0, sample, by [d.entityId, d.callstacksymbol, d.tid] [!*] and

// exclude cmp operations
api where true as seq1, sample, by [d.entityId, d.tid] process.Ext.api.parameters.protection == "RWX" and process.Ext.api.parameters.protect

// Extra DPB Details
sel.process.parent.executable : "C:\Windows\System32\powershell.exe" and
process.Ext.api.summary in ("VirtualProtect", "VirtualAlloc", "NtCreateSection", "NtProtectVirtualMemory", "NtAllocateVirtualMemory", "NtCreateThreadEx", "NtQueueApcThread", "NtSetContextThread", "NtResumeThread", "NtMapViewOfSection", "NtUnmapViewOfSection") and

// Remote Injection
sel.process.parent.executable : "C:\Windows\System32\powershell.exe" and
process.Ext.api.summary in ("VirtualProtectEx", "VirtualAllocEx", "WriteProcessMemory", "NtAllocateVirtualMemoryEx", "NtWriteVirtualMemory", "NtProtectVirtualMemoryEx", "NtCreateThreadEx", "NtQueueApcThreadEx", "NtSetContextThreadEx", "NtResumeThreadEx", "NtMapViewOfSectionEx", "NtUnmapViewOfSectionEx") and

// Memory Scanning
sel.process.parent.executable : "C:\Windows\System32\powershell.exe" and
process.Ext.api.summary in ("NtQueryVirtualMemory", "NtReadVirtualMemory", "NtReadVirtualMemoryEx") and

// Module Stomping
sel.process.Ext.api.name in ("LoadProcessModules", "GetModuleInformation", "EnumerateLoadedModules", "EnumerateLoadedModules64") and
process.parent.executable : ("*\\powershell*\\Microsoft.Management.Infrastructure.CimCmdlets.dll", "*\\Microsoft.PowerShell.Commands.Management.dll", "*\\Microsoft.PowerShell.Commands.Utility.dll", "*\\System.Management.Automation.dll", "*\\System.Management.dll", "*\\System.Net.Http.dll", "*\\System.Net.Sockets.dll", "*\\System.Net.Primitives.dll", "*\\System.Net.Ping.dll", "*\\System.Net.NetworkInformation.dll", "*\\System.Net.Mail.dll", "*\\System.Net.Ldap.dll", "*\\System.Net.Security.dll", "*\\System.Net.Quic.dll", "*\\System.Net.Sockets.dll", "*\\System.Net.WebClient.dll", "*\\System.Net.WebProxy.dll", "*\\System.Net.WebSockets.dll", "*\\System.Net.WebSockets.Client.dll", "*\\System.Net.Requests.dll", "*\\System.Net.Cache.dll", "*\\System.Net.Http.dll", "*\\System.Net.HttpListener.dll", "*\\System.Net.Mail.dll", "*\\System.Net.NameResolution.dll", "*\\System.Net.NetworkInformation.dll", "*\\System.Net.Ping.dll", "*\\System.Net.Primitives.dll", "*\\System.Net.Security
```